

Muneeb Rehman, CT-25092

Question 1.

```
1  #include <iostream>
2  #include <string>
3
4  struct VehicleData
5  {
6      std::string type, make, model, colour, year, totalMileage;
7  };
8
9  class Vehicle
10 {
11 protected:
12     std::string type, make, model, colour, year, totalMileage;
13 public:
14     Vehicle(const VehicleData& d)
15         : type(d.type), make(d.make), model(d.model),
16           colour(d.colour), year(d.year), totalMileage(d.totalMileage) {}
17     virtual void display() = 0;
18     virtual ~Vehicle() {}
19 };
20
21 class GasVehicle : virtual public Vehicle
22 {
23 protected:
24     float fuelCapacity;
25 public:
26     GasVehicle(float fC, const VehicleData& d)
27         : Vehicle(d), fuelCapacity(fC) {}
28 };
29
30 class ElectricVehicle : virtual public Vehicle
31 {
32 protected:
33     float energyStorage;
34 public:
35     ElectricVehicle(float eS, const VehicleData& d)
36         : Vehicle(d), energyStorage(eS) {}
37 };
38
39 class HeavyVehicle : public GasVehicle, public ElectricVehicle
40 {
41 protected:
42     float maxWeight;
43     int totalWheels;
44     float length;
45 public:
46     HeavyVehicle(float mW, int tW, float l, float fC, float eS, const VehicleData& d)
47         : Vehicle(d), GasVehicle(fC, d), ElectricVehicle(eS, d),
48           maxWeight(mW), totalWheels(tW), length(l) {}
49 };
50
51 class HighPerformance : public GasVehicle
52 {
53 protected:
54     float horsePwr;
55     float topSpeed;
56 public:
57     HighPerformance(float hp, float ts, float fC, const VehicleData& d)
58         : Vehicle(d), GasVehicle(fC, d), horsePwr(hp), topSpeed(ts) {}
59 };
60
61 class SportsCar : public HighPerformance
62 {
63     std::string gearbox;
```

```

62 {
63     std::string gearbox;
64     std::string driveSystem;
65 public:
66     SportsCar(std::string gb, std::string ds, float hp, float ts, float fC, const VehicleData& d)
67         : Vehicle(d), HighPerformance(hp, ts, fC, d), gearbox(gb), driveSystem(ds) {}
68     void display() override {
69         std::cout << "=== Sports Car ===\n"
70             << "Type: " << type << "\n"
71             << "Make: " << make << "\n"
72             << "Model: " << model << "\n"
73             << "Colour: " << colour << "\n"
74             << "Year: " << year << "\n"
75             << "Mileage: " << totalMileage << "km\n"
76             << "Fuel Cap: " << fuelCapacity << "L\n"
77             << "Horsepower: " << horsePwr << "hp\n"
78             << "Top Speed: " << topSpeed << "km/h\n"
79             << "Gearbox: " << gearbox << "\n"
80             << "Drive System: " << driveSystem << "\n";
81     }
82 };
83
84 class ConstructionTruck : public HeavyVehicle
85 {
86     std::string cargo;
87 public:
88     ConstructionTruck(std::string cg, float mw, int tw, float l, float fC, float eS, const VehicleData& d)
89         : Vehicle(d), HeavyVehicle(mw, tw, l, fC, eS, d), cargo(cg) {}
90     void display() override {
91         std::cout << "=== Construction Truck ===\n"
92             << "Type: " << type << "\n"
93             << "Make: " << make << "\n"
94             << "Model: " << model << "\n"
95             << "Colour: " << colour << "\n"
96             << "Year: " << year << "\n"
97             << "Mileage: " << totalMileage << "km\n"
98             << "Fuel Cap: " << fuelCapacity << "L\n"
99             << "Energy Store: " << energyStorage << "kwh\n"
100             << "Max Weight: " << maxWeight << "kg\n"
101             << "Wheels: " << totalWheels << "\n"
102             << "Length: " << length << "m\n"
103             << "Cargo: " << cargo << "\n";
104     }
105 };
106
107 class Bus : public HeavyVehicle
108 {
109     int totalSeats;
110 public:
111     Bus(int ts, float mw, int tw, float l, float fC, float eS, const VehicleData& d)
112         : Vehicle(d), HeavyVehicle(mw, tw, l, fC, eS, d), totalSeats(ts) {}
113     void display() override {
114         std::cout << "=== Bus ===\n"
115             << "Type: " << type << "\n"
116             << "Make: " << make << "\n"
117             << "Model: " << model << "\n"
118             << "Colour: " << colour << "\n"
119             << "Year: " << year << "\n"
120             << "Mileage: " << totalMileage << "km\n"
121             << "Fuel Cap: " << fuelCapacity << "L\n"
122             << "Energy Store: " << energyStorage << "kwh\n"
123             << "Max Weight: " << maxWeight << "kg\n"
124             << "Wheels: " << totalWheels << "\n"
125             << "Length: " << length << "m\n"
126             << "Total Seats: " << totalSeats << "\n";
127     }
128 };
129
130 int main()
131 {
132     VehicleData d{"Heavy", "Mercedes", "Citaro", "White", "2022", "0"};
133     Bus bus(50, 18000.0f, 6, 12.0f, 200.0f, 100.0f, d);
134     bus.display();
135 }

```

=== Bus ===

Type: Heavy

Make: Mercedes

Model: Citaro

Colour: White

Year: 2022

Mileage: 0km

Fuel Cap: 200L

Energy Store: 100kWh

Max Weight: 18000kg

Wheels: 6

Length: 12m

Total Seats: 50

Process exited after 0.1399 seconds with return value 0

Press any key to continue . . . |

Question 2.

```
1  #include <iostream>
2  #include <string>
3
4  class Character
5  {
6  protected:
7      std::string name;
8      int level;
9      int health;
10 public:
11     Character(std::string n, int l, int h) : name(n), level(l), health(h) {}
12     virtual void display() = 0;
13     virtual ~Character() {}
14 };
15
16 class Warrior : virtual public Character
17 {
18 protected:
19     int strength;
20     std::string meleeWeapon;
21 public:
22     Warrior(int s, std::string mw, std::string n, int l, int h)
23         : Character(n, l, h), strength(s), meleeWeapon(mw) {}
24     void slash() { std::cout << name << " slashes with " << meleeWeapon << "!\n"; }
25 };
26
27 class Mage : virtual public Character
28 {
29 protected:
30     int intelligence;
31     std::string spellProficiency;
32 public:
33     Mage(int i, std::string sp, std::string n, int l, int h)
34         : Character(n, l, h), intelligence(i), spellProficiency(sp) {}
35     void fireball() { std::cout << name << " casts Fireball!\n"; }
36 };
37
38 class Archer : virtual public Character
39 {
40 protected:
41     int dexterity;
42     std::string rangedWeapon;
43 public:
44     Archer(int d, std::string rw, std::string n, int l, int h)
45         : Character(n, l, h), dexterity(d), rangedWeapon(rw) {}
46     void rapidShot() { std::cout << name << " fires a rapid shot with " << rangedWeapon << "!\n"; }
47     void display() override {
48         std::cout << "=== Archer ===\n"
49             << "Name: " << name << "\n"
50             << "Level: " << level << "\n"
51             << "Health: " << health << "\n"
52             << "Dexterity: " << dexterity << "\n"
53             << "Weapon: " << rangedWeapon << "\n";
54     }
55 };
56
57 class NPC : virtual public Character
58 {
59     std::string dialogue;
60 public:
61     NPC(std::string dlg, std::string n, int l, int h)
62         : Character(n, l, h), dialogue(dlg) {}
63     void speak() { std::cout << name << " says: " << dialogue << "\n"; }
64     void display() override {
65         std::cout << "=== NPC ===\n"
66             << "Name: " << name << "\n"
67             << "Level: " << level << "\n"
68             << "Health: " << health << "\n"
```

```

68         << "Health: " << health << "\n"
69         << "Dialogue: " << dialogue << "\n";
70     }
71 };
72
73 class Mighty : public Warrior, public Mage
74 {
75 public:
76     Mighty(int s, std::string mw, int i, std::string sp, std::string n, int l, i
77         : Character(n, l, h), Warrior(s, mw, n, l, h), Mage(i, sp, n, l, h) {}
78     void display() override {
79         std::cout << "=== Mighty ===\n"
80         << "Name: " << name << "\n"
81         << "Level: " << level << "\n"
82         << "Health: " << health << "\n"
83         << "Strength: " << strength << "\n"
84         << "Weapon: " << meleeWeapon << "\n"
85         << "Intelligence: " << intelligence << "\n"
86         << "Spell Prof: " << spellProficiency << "\n";
87     }
88 };
89
90 int main()
91 {
92     Archer archer(90, "Longbow", "Legolas", 10, 150);
93     archer.display();
94     archer.rapidShot();
95
96     NPC npc("Welcome to the village!", "Villager", 1, 50);
97     npc.display();
98     npc.speak();
99
100    Mighty mighty(95, "Greatsword", 88, "Arcane", "Gandalf", 20, 300);
101    mighty.display();
102    mighty.slash();
103    mighty.fireball();
104 }

```

=== Archer ===

Name: Legolas

Level: 10

Health: 150

Dexterity: 90

Weapon: Longbow

Legolas fires a rapid shot with Longbow!

=== NPC ===

Name: Villager

Level: 1

Health: 50

Dialogue: Welcome to the village!

Villager says: Welcome to the village!

=== Mighty ===

Name: Gandalf

Level: 20

Health: 300

Strength: 95

Weapon: Greatsword

Intelligence: 88

Spell Prof: Arcane

Gandalf slashes with Greatsword!

Gandalf casts Fireball!

Question 3.

```
1  #include <iostream>
2
3  class Account
4  {
5  protected:
6      double balance;
7  public:
8      Account(double b = 0) : balance(b) {}
9      virtual void deposit(double amount) { balance += amount; }
10     virtual void withdraw(double amount) { balance -= amount; }
11     void checkBalance() { std::cout << "Balance: Rs." << balance << "\n"; }
12 };
13
14 class InterestAccount : virtual public Account
15 {
16 protected:
17     double interest; // as a percentage
18 public:
19     InterestAccount(double b = 0, double i = 30.0) : Account(b), interest(i) {}
20     void deposit(double amount) override {
21         double withInterest = amount + (amount * interest / 100);
22         balance += withInterest;
23         std::cout << "Deposited Rs." << amount << " + Rs." << (amount * interest / 100) << " interest\n";
24     }
25 };
26
27 class ChargingAccount : virtual public Account
28 {
29 protected:
30     double fee;
31 public:
32     ChargingAccount(double b = 0, double f = 25.0) : Account(b), fee(f) {}
33     void withdraw(double amount) override {
34         balance -= (amount + fee);
35         std::cout << "Withdrew Rs." << amount << " + Rs." << fee << " fee\n";
36     }
37 };
38
39 class ACI : public InterestAccount, public ChargingAccount
40 {
41 public:
42     ACI(double b = 0, double i = 30.0, double f = 25.0)
43         : Account(b), InterestAccount(b, i), ChargingAccount(b, f) {}
44
45     void transfer(double amount, Account& target) {
46         withdraw(amount);
47         target.deposit(amount);
48         std::cout << "Transferred Rs." << amount << "\n";
49     }
50     void transfer(double amount, InterestAccount& target) {
51         withdraw(amount);
52         target.deposit(amount);
53         std::cout << "Transferred Rs." << amount << " to Interest Account\n";
54     }
55     void transfer(double amount, ChargingAccount& target) {
56         withdraw(amount);
57         target.deposit(amount);
58         std::cout << "Transferred Rs." << amount << " to Charging Account\n";
59     }
60     void deposit(double amount) override { InterestAccount::deposit(amount); }
61     void withdraw(double amount) override { ChargingAccount::withdraw(amount); }
62 };
63
64 int main()
65 {
66     ACI myAccount(1000.0);
67     std::cout << "-- Initial --\n";
68     myAccount.checkBalance();
```

```

64 int main()
65 {
66     ACI myAccount(1000.0);
67     std::cout << "-- Initial --\n";
68     myAccount.checkBalance();
69
70     std::cout << "\n-- Deposit Rs.500 --\n";
71     myAccount.deposit(500);
72     myAccount.checkBalance();
73
74     std::cout << "\n-- Withdraw Rs.200 --\n";
75     myAccount.withdraw(200);
76     myAccount.checkBalance();
77
78     Account other(500);
79     std::cout << "\n-- Transfer Rs.100 to another account --\n";
80     myAccount.transfer(100, other);
81     myAccount.checkBalance();
82 }

```

-- Initial --

Balance: Rs.1000

-- Deposit Rs.500 --

Deposited Rs.500 + Rs.150 interest

Balance: Rs.1650

-- Withdraw Rs.200 --

Withdrew Rs.200 + Rs.25 fee

Balance: Rs.1425

-- Transfer Rs.100 to another account --

Withdrew Rs.100 + Rs.25 fee

Transferred Rs.100

Balance: Rs.1300

Question 4.

```
1  #include <iostream>
2  #include <string>
3
4  class Media
5  {
6  protected:
7      std::string title;
8      bool borrowed;
9  public:
10     Media(std::string t) : title(t), borrowed(false) {}
11     void borrow() {
12         if (!borrowed) { borrowed = true; std::cout << title << " borrowed.\n"; }
13         else
14             std::cout << title << " is already borrowed.\n";
15     }
16     void returnMedia() {
17         if (borrowed) { borrowed = false; std::cout << title << " returned.\n"; }
18         else
19             std::cout << title << " was not borrowed.\n";
20     }
21     virtual void display() = 0;
22     virtual ~Media() {}
23 };
24
25 class Book : public Media
26 {
27     std::string author;
28 public:
29     Book(std::string t, std::string a) : Media(t), author(a) {}
30     void display() override {
31         std::cout << "=== Book ===\n"
32             << "Title: " << title << "\n"
33             << "Author: " << author << "\n"
34             << "Status: " << (borrowed ? "Borrowed" : "Available") << "\n";
35     }
36 };
37
38 class Magazine : public Media
39 {
40     int issueNumber;
41 public:
42     Magazine(std::string t, int issue) : Media(t), issueNumber(issue) {}
43     void display() override {
44         std::cout << "=== Magazine ===\n"
45             << "Title: " << title << "\n"
46             << "Issue: " << issueNumber << "\n"
47             << "Status: " << (borrowed ? "Borrowed" : "Available") << "\n";
48     }
49 };
50
51 class DVD : public Media
52 {
53     std::string director;
54 public:
55     DVD(std::string t, std::string d) : Media(t), director(d) {}
56     void display() override {
57         std::cout << "=== DVD ===\n"
58             << "Title: " << title << "\n"
59             << "Director: " << director << "\n"
60             << "Status: " << (borrowed ? "Borrowed" : "Available") << "\n";
61     }
62 };
63
64 int main()
```

```
62 int main()
63 {
64     Book book("The Hobbit", "J.R.R. Tolkien");
65     Magazine mag("National Geographic", 245);
66     DVD dvd("Inception", "Christopher Nolan");
67
68     book.display();
69     book.borrow();
70     book.borrow(); // try borrowing again
71     book.returnMedia();
72
73     std::cout << "\n";
74     mag.display();
75     mag.borrow();
76
77     std::cout << "\n";
78     dvd.display();
79     dvd.borrow();
80     dvd.returnMedia();
81 }
```

=== Book ===

Title: The Hobbit

Author: J.R.R. Tolkien

Status: Available

The Hobbit borrowed.

The Hobbit is already borrowed.

The Hobbit returned.

=== Magazine ===

Title: National Geographic

Issue: 245

Status: Available

National Geographic borrowed.

=== DVD ===

Title: Inception

Director: Christopher Nolan

Status: Available

Inception borrowed.

Inception returned.